Tecnologías de Registro Distribuido y Blockchain (BC)



Práctica 1 - Ejercicio 5

Bingo

Martín López Villaverde
Rubén Barba Paz

1. Análisis y definición del escenario, debe requerir la funcionalidad proporcionada por una blockchain pública como Ethereum.

El escenario propuesto para su aplicación en una blockchain pública como Ethereum es el **bingo**.
El bingo es un juego de azar en el que varios jugadores compran cartones con números y un presentador extrae bolas numeradas de un bombo, una a una. El ganador es aquel jugador que antes cante “bingo”, es decir, que complete su cartón. Recibe un premio financiado con el dinero aportado por los participantes.

Este clásico y sencillo juego, sin embargo, cuenta con numerosos problemas en lo que a confianza y gestión del dinero respecta:

- **Falta de confianza en la extracción de los números.** Los jugadores no pueden comprobar que las bolas salen realmente al azar, sino que confían en el organizador y su extracción. Este podría en realidad manipular el bombo, favorecer a un jugador concreto o repetir y omitir números sin que nadie lo detecte.
- **Falta de transparencia y trazabilidad.** A pesar de que, por ley, exista un registro de los números que han salido, es imprescindible asegurar que este registro no puede ser alterado. Si surge una disputa sobre qué números han salido y en qué orden, debe existir un mecanismo de acceso público que permita verificar y asegurar el historial de números extraídos.
- **Gestión opaca del dinero.** El organizador recoge el dinero de las apuestas y paga el premio. Los jugadores confían en que el bote existe, en que se reparte correctamente y en que el pago llegará.
- **Validación manual del bingo.** La comprobación de un cartón ganador la realiza una persona, lo que deja margen para errores, fraudes o favoritismos.
- **Dependencia de un intermediario.** Todo el sistema descansa en una única entidad que puede fallar, actuar de mala fe o desaparecer con el dinero.

En resumen, el problema principal del bingo es la necesidad de **confiar en un tercero**. Esto es justo lo que permitiría eliminar una blockchain pública como Ethereum. Con ella, los jugadores no necesitan confiar en una persona, pues les basta poder leer el código del contrato, que es público y determinista. En concreto, para cada carencia del bingo tradicional, una blockchain aporta una solución eficaz:

- **¿Desconfianza en las bolas extraídas?** → Cada número queda registrado en una cadena pública. Además, las reglas de extracción están claramente definidas en el código. Todo el mundo las conoce, no hay lugar a fraudes de extracción.
- **¿Posibilidad de alterar el historial de números extraídos?** → La blockchain es inmutable, es decir, impide modificar y mucho menos borrar los números extraídos.
- **¿Gestión opaca del bote?** → El contrato inteligente custodia los fondos. Nadie puede retirarlos fuera de las reglas programadas.
- **¿Validación manual de cartones?** → La comprobación del bingo ya no la realiza un único tercero, sino que es ejecutada de forma automática y determinista en el contrato.
- **¿Dependencia de un intermediario?** → El contrato se ejecuta de forma descentralizada, sin que ninguna parte pueda alterar su comportamiento

Por tanto, se propone un **bingo descentralizado** implementado como contrato inteligente en Ethereum.
El flujo de una partida en este bingo comienza con la creación de una partida en la que se establece un precio de cartón y plazo de inscripción, tras lo cual los jugadores pagarían ether para recibir un cartón. Estos pagos se acumulan en el bote que custodia el contrato.
Al extraer los números (bolas) uno a uno y sin repetición, cada uno de estos números extraídos queda registrado en la blockchain y es verificable por cualquiera.
Cuando un jugador completa su cartón, lo comunica al contrato, que verifica automática e instantáneamente que su "bingo" (números que han salido) es válido. Este jugador ganador validado recibe el premio (transferido por el contrato) sin intervención de terceros.

Para dotar al juego de un incentivo adicional, se considera un **mecanismo de recompensas** basado en un **premio extra por bingo rápido**, es decir, si un jugador canta bingo antes de que se haya extraído un número determinado de bolas (X rondas), recibe bonus además del premio base. De esta forma, se incentiva la participación haciendo el juego más atractivo desde el punto de vista económico, pudiendo verificar igualmente que el bingo ha sido cantado por debajo del número de rondas estipulado.

2. Diseño

- **Definir un caso de uso (lógica de negocio, automatización de transacciones de determinados aspectos).**
- **Definir el contenido del contrato inteligente, específicamente sus datos, funciones que operan sobre los datos, reglas de operación, entre otros.**
- **Definir los usuarios del sistema (e.g., diagrama de casos de uso).**

Caso de uso
El contrato relacionado con el bingo gestiona partidas de bingo completas, desde la inscripción y custodia del bote hasta la extracción verificable de bolas, la validación automática del bingo y el reparto del premio. El organizador solo crea la partida y aporta la semilla de aleatoriedad. No puede tocar el dinero ni influir en quién gana.
Este bingo emplea bolas del 1 al 90 junto con cartones de 15 números distintos con una proporción de 3 filas x 9 columnas, es decir, un bingo clásico.

En relación a la automatización, el contrato es capaz de ejecutar por sí mismo (sin intermediarios) las siguientes acciones:

- El cobro de cada cartón y su acumulación en el bote.
- La generación de cada cartón en el momento de la compra.
- La extracción de bolas sin repetición.
- La comprobación del cartón cuando un jugador canta bingo.
- El cálculo del premio (base y bonus por bingo rápido) y su asignación al ganador.
- Los reembolsos automáticos si la partida se cancela, o, en caso contrario, el pago correspondiente al ganador.

Contenido del contrato inteligente (datos, funciones, reglas)
En relación a los datos, se pueden distinguir varios tipos de estos, desde constantes, estados, structs, mappings… A continuación se expone cada uno de ellos:

- Constantes.

    - *NUM_BOLAS* → 90 bolas con las que se jugará el bingo.
    - *NUMS_POR_CARTON* → 15 números que tendrá cada cartón.
    - *PORC_ORGANIZADOR* → 10% del bote total es la comisión del organizador.
    - *PORC_BONUS* → 20% del bote total es lo que se reserva para el bonus.
- Estados.
    - *enum Estado → Inscripcion/Cerrada/EnCurso/Finalizada/Cancelada*
- Structs.
    - *Partida*:
        - organizador → Dirección del que creó la partida.
        - estado → Estado actual.
        - precioCarton → Precio en wei.
        - finInscripcion → Bloquea la inscripción cuando se llega al límite.
        - topeBonus (X) → Máximo de bolas para cobrar el bonus.
        - bolasExtraidas → Contador de bolas extraídas.
        - numerosExtraidos → Array de bits que indican las bolas ya salidas.
        - ultimoBloqueBola → Bloque en la blockchain de la última bola extraída.
        - compromiso → Hash de la semilla del organizador (commit).
        - semilla → semilla final, tras la revelación.
        - bote → Bote total recaudado.
        - numCartones → Número de cartones vendidos (en juego).
    - *Carton*:
        - address dueño → Dirección del dueño del cartón.
        - uint96 mascara → Bits del cartón (uno por bola de la 1 a la 90)
- Mappings.
    - *partidas[idPartida]* → Relaciona cada partida con su ID.
    - *cartones[idPartida][idCarton]* → Relaciona cada cartón con su ID y con el ID de la partida en que fue jugado.
    - *pendiente[address]* → Saldo retirable de cada usuario.

En cuanto a las funciones que emplea el contrato inteligente, estas son las siguientes:

- *crearPartida(precio, plazo, X, compromiso)* → Ejecutada por el organizador. Crea la partida y guarda el hash de su semilla. Emite evento.
- *comprarCarton(idPartida) payable* → Ejecutada por el jugador. Valida el pago, genera el cartón y suma al bote. Emplea payable para operar con transacciones de Ether. Emite evento.
- *revelarSemilla(idPartida, secreto)* → Ejecutada por el organizador. Comprueba que *keccak256(secreto)* == *compromiso*, calcula la semilla final y configura el estado a *EnCurso*. Emite evento.
- *calcularCompromiso(secreto)* → Ejecutada por el organizador dentro de *revelarSemilla* y durante las pruebas para obtener su compromiso. En un uso real, el organizador calcularía el hash fuera de la cadena (en su propio ordenador, por ejemplo). Si el secreto se enviara a una función del contrato, este quedaría visible en la transacción y dejaría de ser secreto. Es por esto que esta función es únicamente usada durante las pruebas.
- *sacarBola(idPartida)* → Ejecutada por cualquiera con un lapso mínimo de tiempo (ver reglas). Extrae la siguiente bola, sin repetición, y emite un evento. Emite evento.

- *cantarBingo(idPartida, idCarton)* → Ejecutada por el jugador. Verifica el cartón, asigna el premio (más bonus si aplica) y finaliza. Emite evento.
- *cancelarPartida(idPartida)* → Ejecutada por cualquiera. Cancela la partida si se cumple una condición de fallo establecida. Emite evento.
- *reclamarReembolso(idPartida, idCarton)* → Ejecutada por el jugador. Devuelve el precio del cartón si la partida se canceló.
- *retirar()* → Ejecutada tanto por el organizador como por el jugador. Transfiere el saldo pendiente a quien lo pide.
- Funciones *view* → Aquí se engloban todas aquellas funciones (similares entre sí) que pueden ser ejecutadas por cualquiera y permiten consultar el estado, las bolas extraídas, los cartones, etc.

De cara a la optimización de gas, se han empleado numerosos eventos (baratos en gas) y que permiten auditar el historial, algo imprescindible para un bingo. Tal y como indica su nombre, estos son *PartidaCreada, CartonComprado, SemillaRevelada, BolaExtraida(idPartida, numero, orden), BingoCantado* y *PartidaCancelada.*

Para finalizar con el contenido del contrato inteligente cabe mencionar algunas reglas de operación que se han establecido para hacer este bingo no solo más fiel al clásico, sino también más seguro y adaptarlo así a la blockchain:

- Solo se compran cartones mientras el estado sea *Inscripcion* y no se haya alcanzado *finInscripcion*. El pago debe ser exactamente *precioCarton*.
- Si al cerrar la inscripción hay menos de 2 cartones, la partida puede cancelarse y se reembolsa.
- Para garantizar la transparencia del sistema, solo el organizador puede revelar la semilla dentro de un plazo límite tras el cierre. Si no lo hace, la partida se puede cancelar públicamente y el contrato devuelve de forma íntegra los fondos a los jugadores.
- Debe pasar un mínimo de bloques entre bolas (es decir, entre llamadas a *sacarBola()*) para asegurar que los usuarios tengan tiempo de cantar bingo.
- Una bola nunca se repite. Se comprueba contra la máscara *numerosExtraidos* (óptima en gas y diseñada para esto).
- Solo el dueño del cartón puede cantar bingo con su cartón, con la partida en el estado *EnCurso*, y el cartón se verifica con *(mascara & numerosExtraidos) == mascara*. Si varios jugadores cantan bingo, gana la primera transacción válida incluida en un bloque.
- En relación al bonus por bingo rápido: si *bolasExtraidas* <= *X* (siendo X el número de rondas/bolas jugadas) en el momento del canto, el ganador recibe también el bonus.
- El contrato nunca envía ether directamente. Acredita el saldo en pendiente y el usuario lo retira con retirar().
- Tras un bingo válido el estado pasa a *Finalizada* y no se aceptan más acciones.
- El reparto del bote se realizará de la siguiente manera:
    - 10% → Comisión del organizador.
    - 20% → Reserva de bonus para el ganador si este canta antes de X bolas.
    - 70% → Premio base para el ganador.

  Es importante destacar que, si nadie cobra el bonus, este es cobrado por el organizador, recibiendo este así el 30% del bote en estos casos.

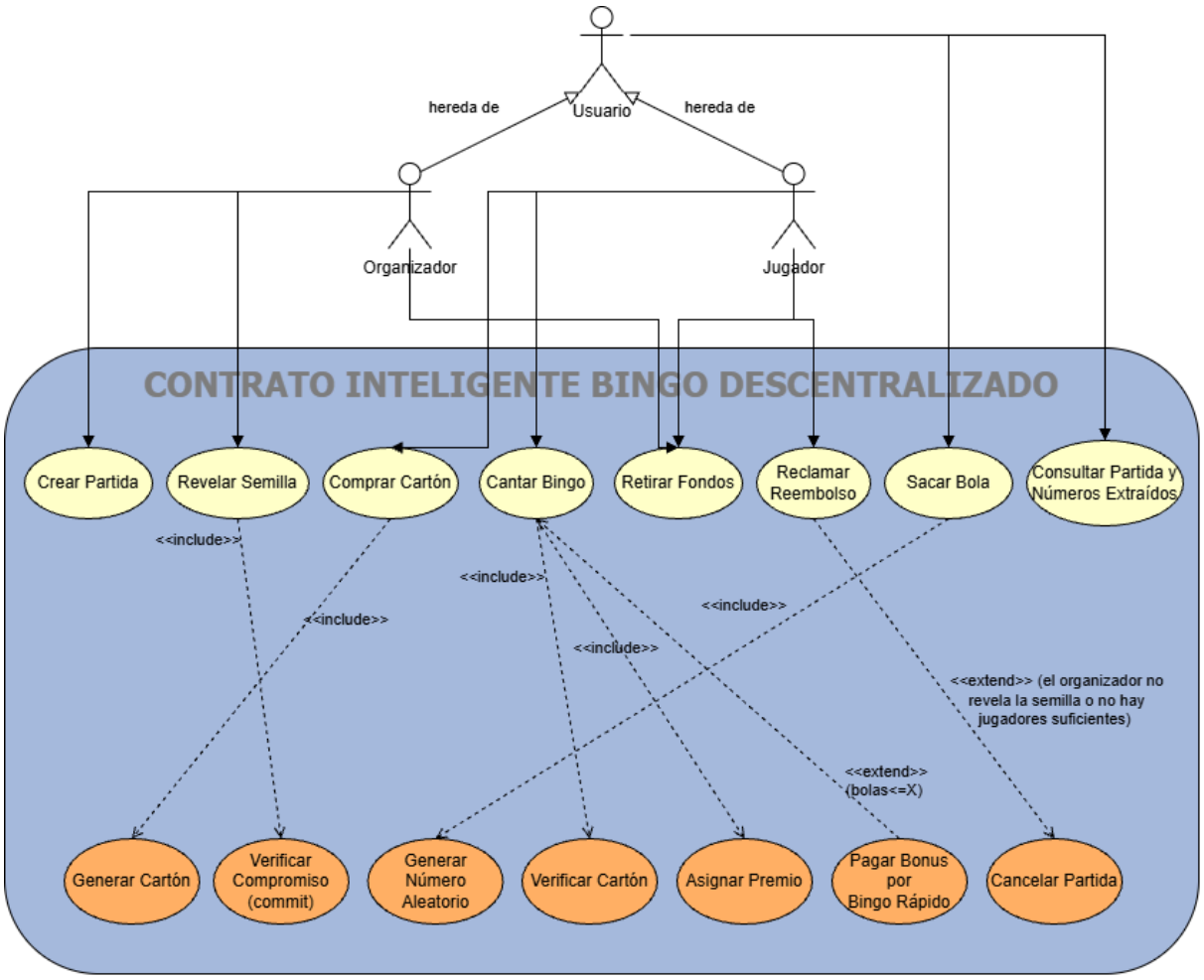
Usuarios del sistema

En este sistema de bingo se pueden distinguir tres actores principales:

- Usuario (cualquier cuenta): Define un actor general. Cualquier dirección de Ethereum puede consultar el estado de un bingo. Además, en caso de que el organizador no haya sacado bola en el tiempo reglamentario, cualquier usuario podrá también llamar a dicha función.
- Organizador: Hereda de usuario. Crea la partida, revela las semillas y saca las bolas.
- Jugador: Hereda de usuario. Compra cartones, canta bingo y cobra.

Además, un mismo usuario puede ser a la vez organizador y jugador.

A continuación, se muestra un diagrama de casos de uso que representa cada acción con su actor correspondiente:



3. Implementación

- **Programar el código del contrato inteligente utilizando el conocimiento adquirido en los ejercicios previos y utilizar técnicas de optimización de gas.**

El código del contrato inteligente se puede encontrar en el siguiente repositorio de GitHub:
https://github.com/ByRuBiX12/CiberBC_Ruben_Martin/tree/main

En relación a la optimización de gas, se emplean ciertas técnicas que permiten reducir su gasto innecesariamente.

En primer lugar, se considera el **orden de las variables en los *structs***, ordenando los campos de estos en *slots* de 32 bytes. De esta forma, con menos *slots* (y menos gas) almacenamos la información eficientemente.
Además, un punto muy importante es el almacenamiento de los números del cartón. En lugar de guardar los 15 números en un array (el cual requeriría 15 escrituras), estos son almacenados en una **máscara** (*uint96*, un bit por número), realizando así una única escritura. Esto sucede de igual forma con las bolas extraídas.
Esta máscara es muy útil a la hora de **verificar el bingo**, pues ahorra bucles y permite comprobar los 15 números con una sola operación *AND*, comprobando la máscara del cartón (bits del cartón a 1) con la máscara del bingo (bits de bolas extraídas a 1).

A la hora de leer información y ejecutar operaciones, también se ha optimizado en la medida de lo posible el gasto del gas.
En todas las funciones se emplean **punteros *storage*** en lugar de copiar *structs* a *memory* para leer y escribir únicamente los campos necesarios. Estas funciones se han declarado con visibilidad *external*, lo cual es más barato que *public*.
Finalmente, otra medida que limita el gasto de gas se encuentra en la función *cancelarPartida()*, la cual solo cambia el estado de la partida, tras lo que cada jugador por separado debe reclamar su reembolso. El coste no crece con el número de jugadores y la función no puede bloquearse por falta de gas.

Por otro lado, también se ha tenido en cuenta la seguridad a la hora de ciertas transacciones susceptibles de poder ser ejecutadas de forma fraudulenta.
La principal vía de ataque en este contrato es la relacionada con la transferencia de fondos al usuario. Así pues, el contrato nunca envía ether en *cantarBingo()* ni en *reclamarReembolso().*
De hecho, en *reclamarReembolso()* el contrato primero anula el cartón y luego devuelve el precio al jugador. De esta forma, se evitan ataques de doble reembolso en los que un usuario pueda ejecutar *reclamarReembolso()* indefinidamente hasta agotar los fondos del contrato.

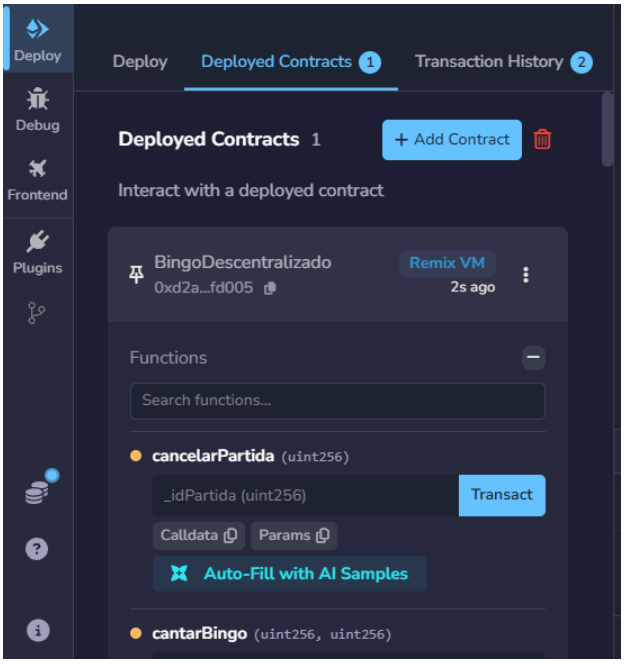
Bajo esta premisa, *retirar()* establece primero a 0 el saldo pendiente del jugador antes de transferirlo.

4. Pruebas

- **Realizar pruebas del contrato inteligente en Remix IDE.**

A continuación, se expone cada prueba (fallida y exitosa) realizada junto a un bingo completo. Un breve vídeo con la realización de ciertas pruebas también está disponible.

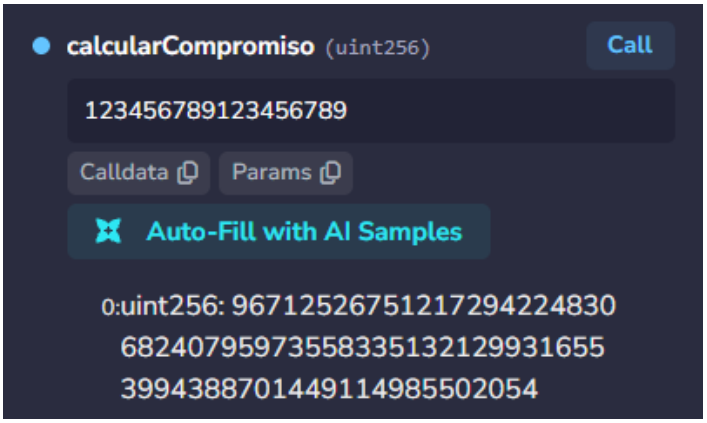
En primer lugar, se compila y se despliega el contrato sin errores:



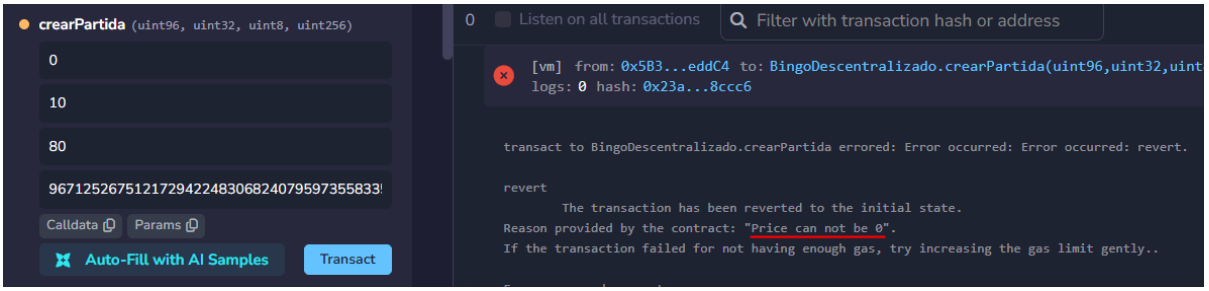
Siguiendo el flujo lógico del bingo, comenzamos las pruebas por la **creación de la partida**.

Para ello, primero creamos un **compromiso** mediante *calcularCompromiso()*. Tal como se indicó en el punto 2, esta función es únicamente empleada durante las pruebas, pues el secreto queda almacenado públicamente en las transacciones, lo cual no debería ocurrir en un uso real, donde el organizador debería calcular este hash localmente y no mediante *calcularCompromiso()*.

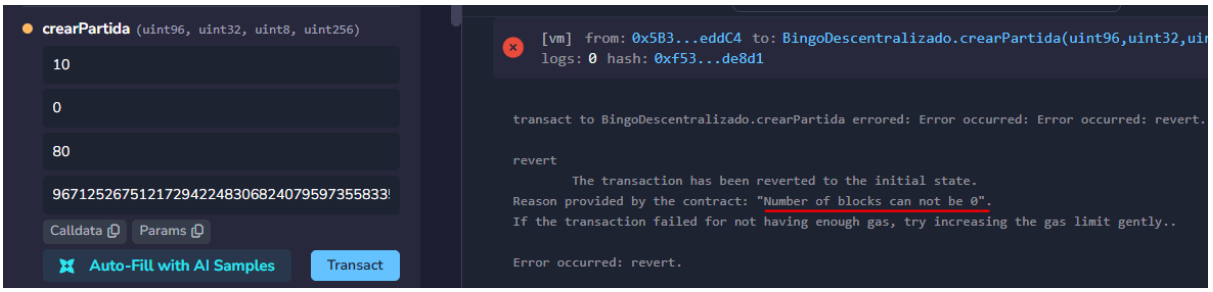
Con el secreto *123456789123456789* obtenemos el compromiso *96712526751217294224830682407959735583351321299316553994388701449114985502054*:



Ahora, con este compromiso, ya podemos crear la partida. Si tratamos de crear una partida con *precio=0*:



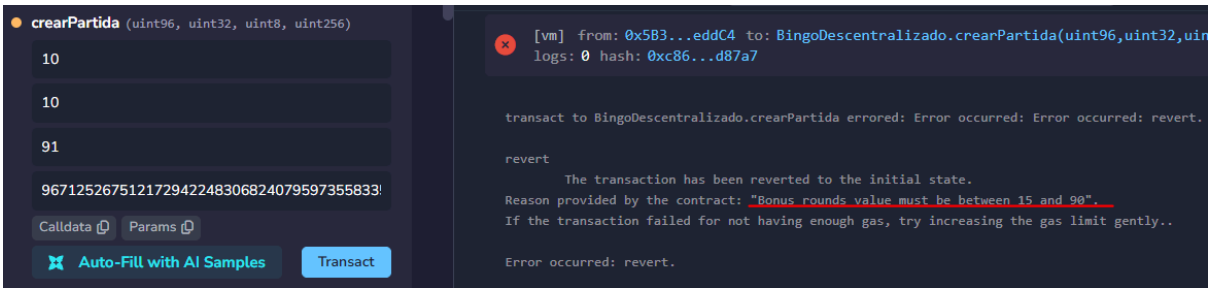
O con *plazo=0*:



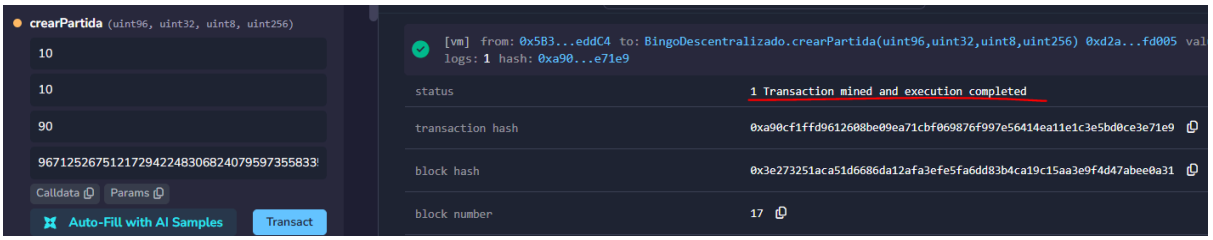
Obtenemos un error alertando de que esto no es posible. De igual forma, si el número de bolas para considerarse bingo bonus es menor que 15:



O mayor que 90:



Si todos los campos son correctos, creamos una partida con *precio=10*, *plazo=10*, *X=90* (bolas para bonus) y el *compromiso* creado anteriormente.



El *id* de esta partida es, tal como indica la salida, 0:



Así, podemos consultar el mapping *partidas* para conocer más datos sobre ella:

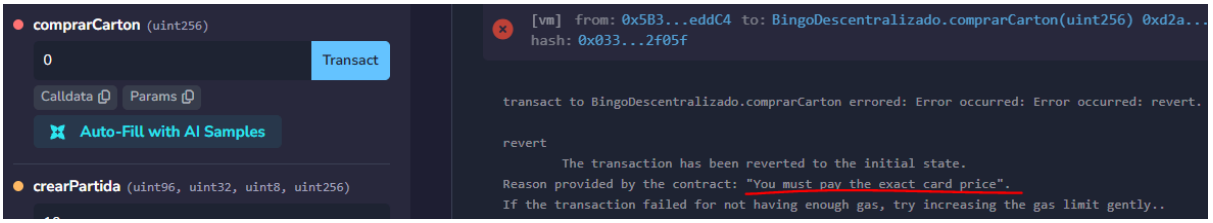


Por tanto, *siguienteId* nos muestra, tal como era esperado, el *id* 1 para la siguiente partida que sea creada:

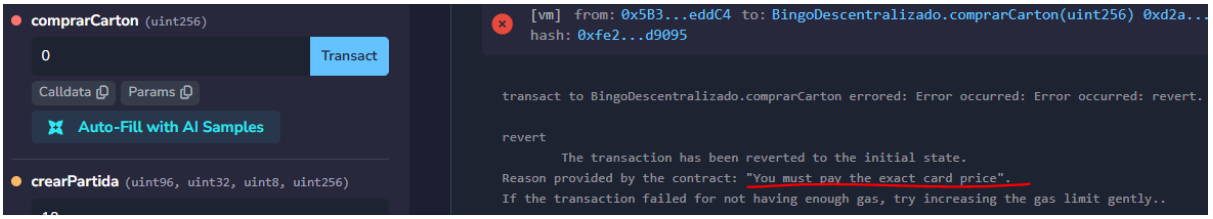


A continuación, se puede realizar la **compra de cartones**.
Hay varias situaciones en las que la compra de un cartón no se debería efectuar, como son las siguientes.
Pagar menos o más que el precio establecido por la partida (*id=0, precio=10*):

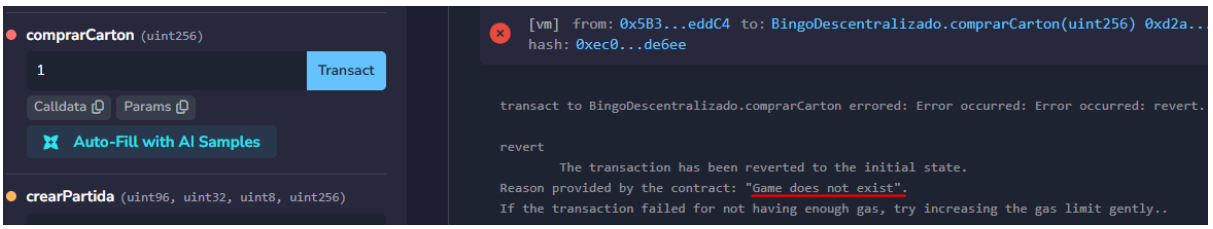




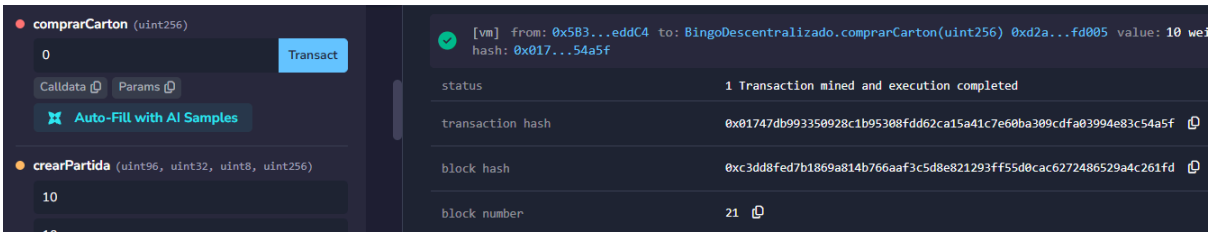




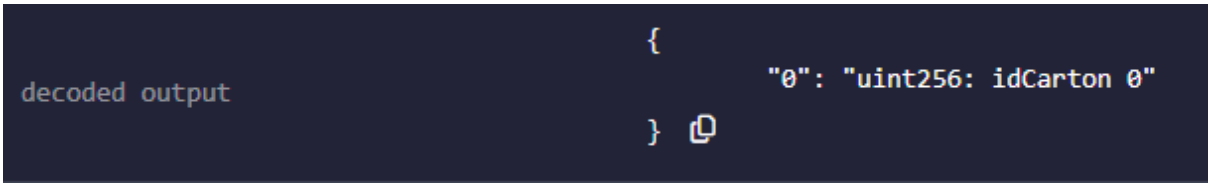
Aún introduciendo el precio correcto (10), se comprueba igualmente que la partida para la que se compra un cartón exista:



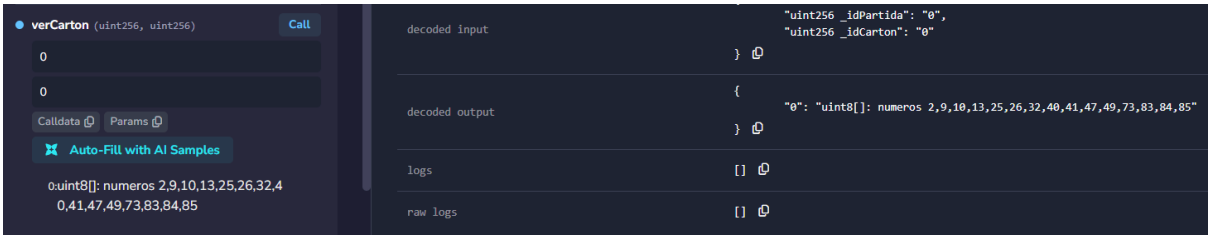
Si ambos valores son válidos, se produce la compra del cartón:



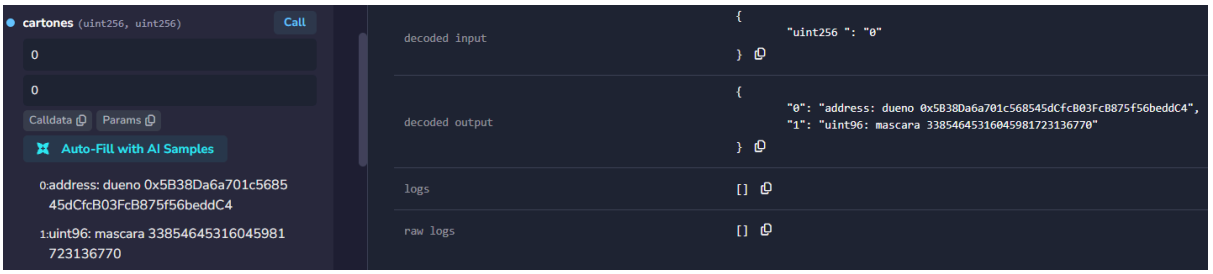
La salida nos muestra el *id* del cartón recién creado:



Podemos ver los números del cartón mediante una llamada a *verCarton()* indicando el *id* de la partida y el *id* del cartón:



Mediante el mapping *cartones* podemos comprobar el dueño y la máscara:

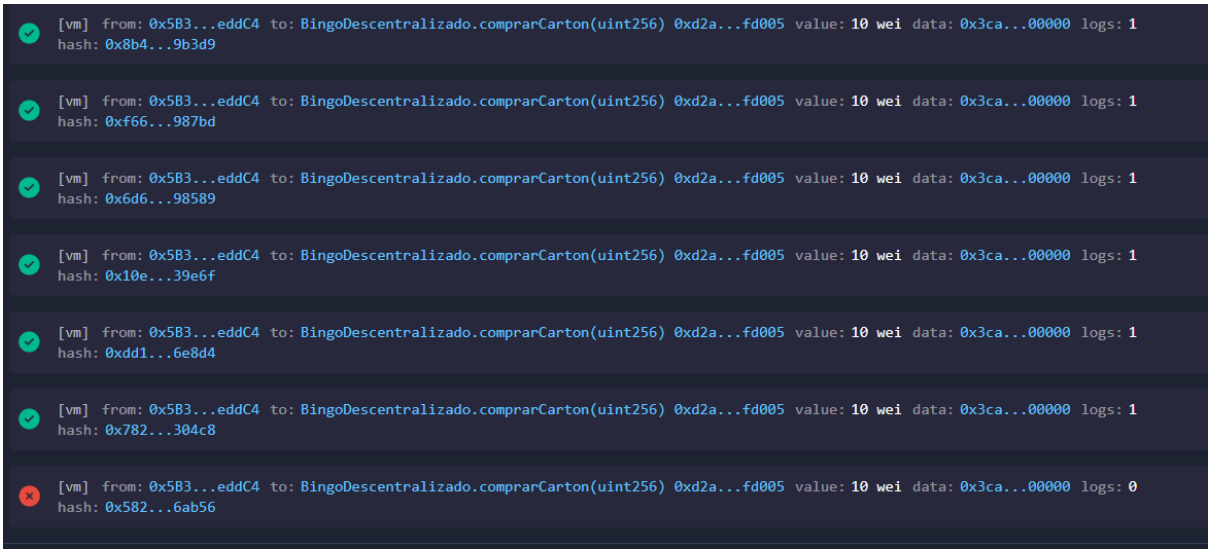


La máscara aparece expresado como un decimal sin sentido aparente:
338546453160459817231367 70
Sin embargo, si lo convertimos a binario obtenemos:

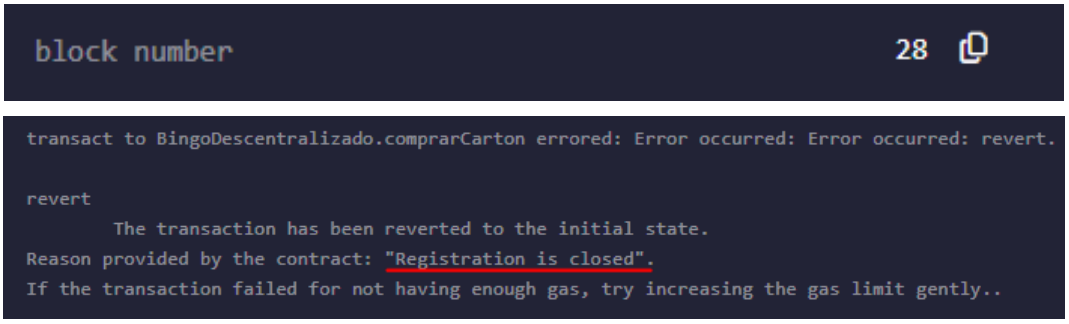
1110000000 ← [85-76]
0010000000 ← [75-66]
0000000000 ← [65-56]
0000001010 ← [55-46]
0000110000 ← [45-36]
0001000001 ← [35-26]
1000000000 ← [25-16]
0010011000 ← [15-06]
00010 ← [05-06]

Esa es precisamente la máscara con la que trabaja el contrato. Una **máscara binaria** en la que la posición de los "**1**" representa los **números del cartón**.

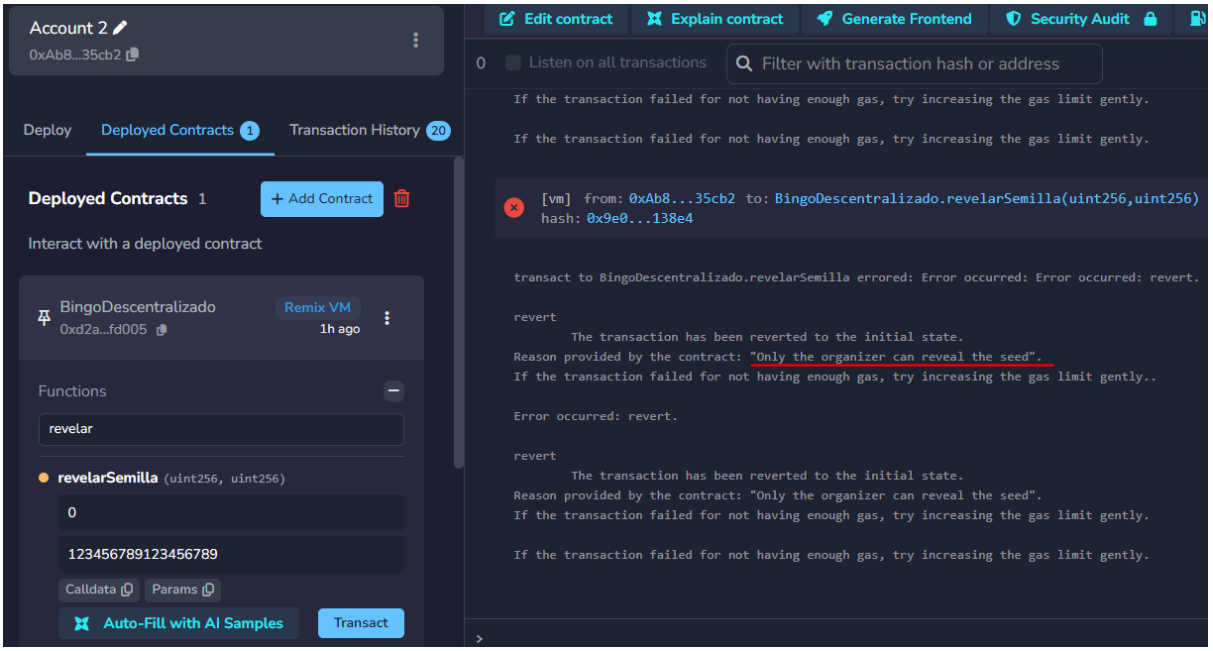
Para que la partida sea válida debe haber al menos 2 cartones. Recordemos que el último bloque minado (transacción exitosa de *comprarCarton()*) fue el número 21. Dado que el plazo (límite de bloques) establecido de la partida era de 10, y el bloque en el que fue creada fue el 17, nos deja con 6 bloques a minar hasta que la partida llegue a su estado de *finInscripcion* tras el que no debería de aceptar más cartones.
Para aumentar este número de bloques podemos crear partidas, comprar cartones… cualquier acción que conlleve minar un bloque.
Así, compraremos 6 cartones, esperando recibir un error a la hora de comprar el cartón nº7:



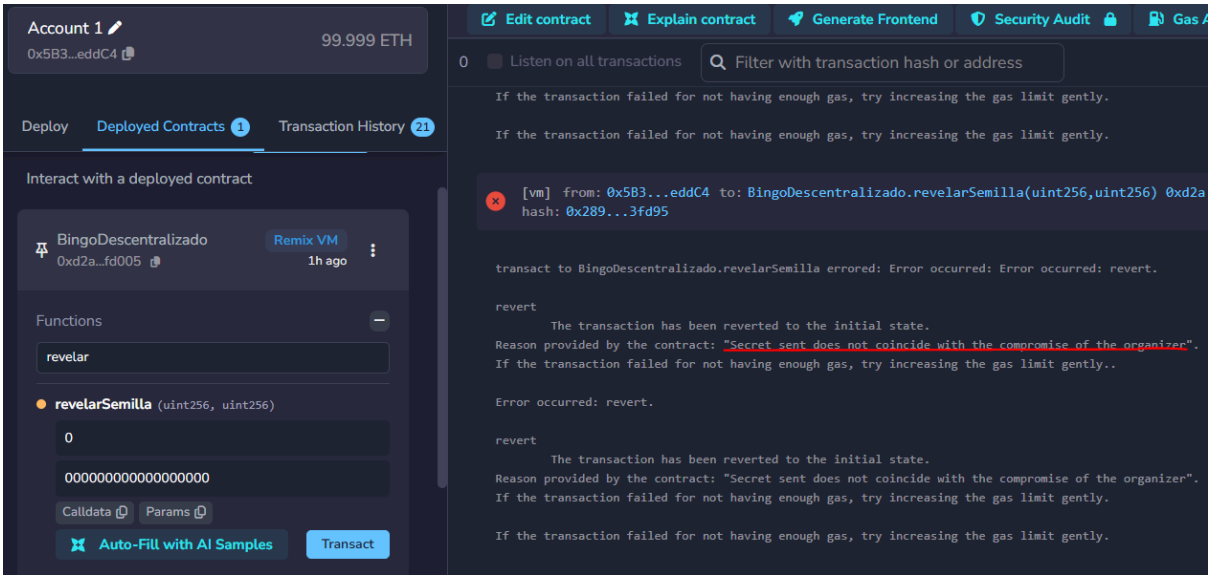
La última transacción pertenece al bloque nº28 y, para entonces, el registro ya se ha cerrado:



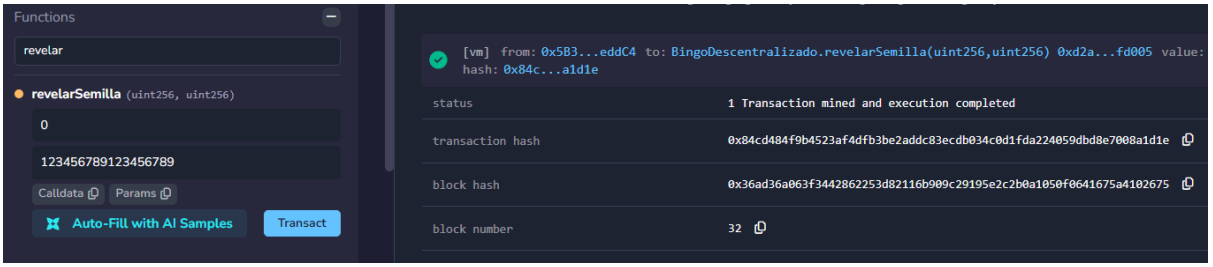
Este es el punto ideal para probar la **revelación de la semilla**, algo que solo puede hacer el organizador en este momento y que garantiza la transparencia del juego.
En primer lugar, si alguien (*Account 2)*que no es el organizador (*Account 1)* trata de *revelarSemilla()*, este no es capaz:



Aún siendo el organizador, el secreto debe coincidir con el que fue empleado para crear la partida:



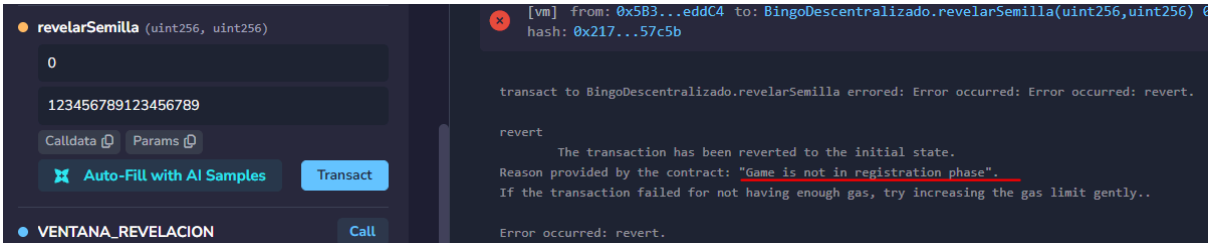
Si los valores son correctos se revela la semilla:



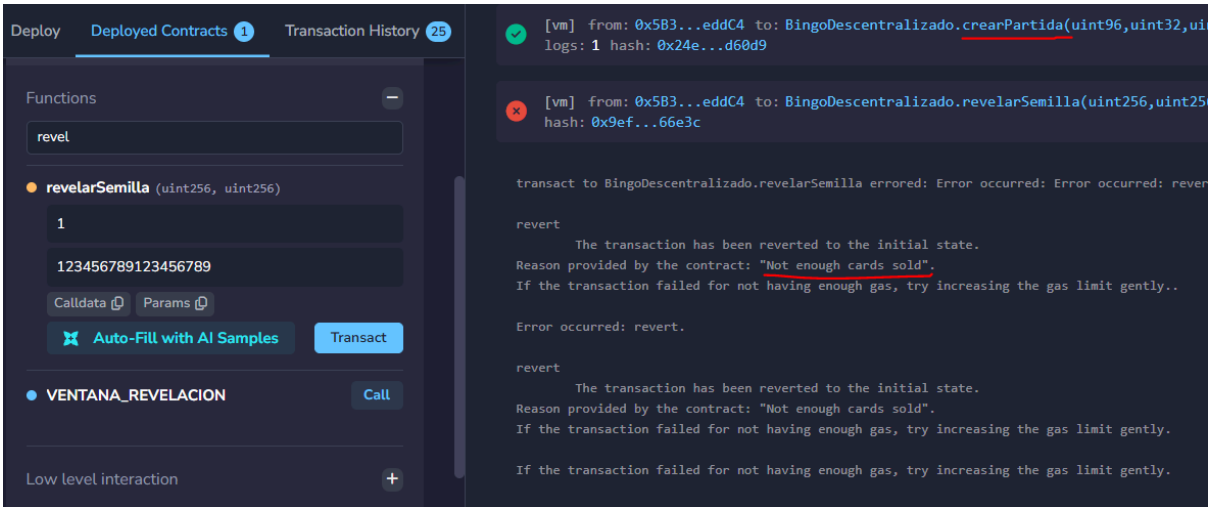
Y queda establecida en la partida con *id=0*, la cual ya muestra ya muestra un *estadoActual=1*, es decir, *EnCurso*; 7 cartones comprados y un bote de 70 (10 wei por cartón):


```
{
        "0": "address: organizador 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4",
        "1": "uint8: estadoActual 1",
        "2": "uint8: topeBonus 90",
        "3": "uint8: bolasExtraidas 0",
        "4": "uint32: finInscripcion 27",
        "5": "uint32: ultimoBloqueBola 32",
        "6": "uint96: numerosExtraidos 0",
        "7": "uint96: precioCarton 10",
        "8": "uint32: numCartones 7",
        "9": "uint256: bote 70",
        "10": "uint256: compromiso 96712526751217294224830682407959735583351321299316553994388701449114985502054",
        "11": "uint256: semilla 260267213301766066965353765536979003104488677202742945565158016295277143502 63"
}
```

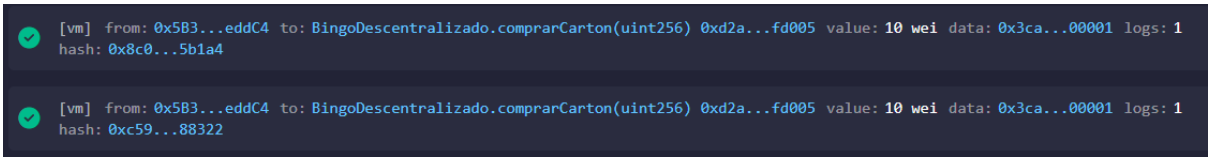

Si se quisiera revelar de nuevo la semilla, la partida ya no se encontraría en la fase permitida para revelarla:



Hay otros casos en los que la semilla tampoco debería poder ser revelada. Esto sucede cuando se trata de revelar la semilla de una partida que tiene menos de 2 cartones, o cuando no se ha llegado al bloque de referencia, es decir, cuando no se ha alcanzado el bloque establecido por el límite de *finInscripcion*.
Para probar esto, es necesario crear una nueva partida con *plazo=5* que tendrá *id=1.* Si tratamos de revelar la semilla nada más crearla, nos advierte de que no se han vendido suficientes cartones (2, en este caso):



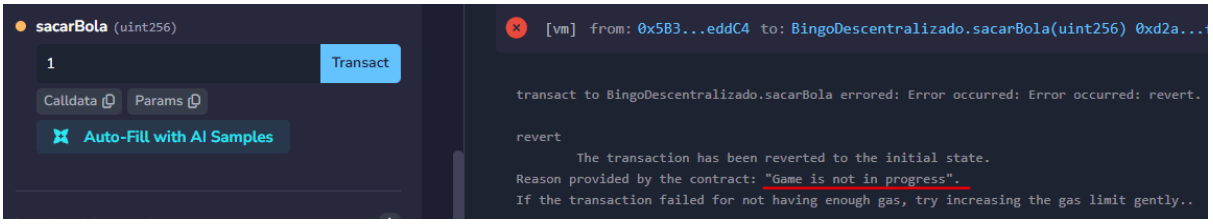
Por tanto, compramos 2 cartones para la partida con *id=1* y volvemos a tratar de revelar la semilla:



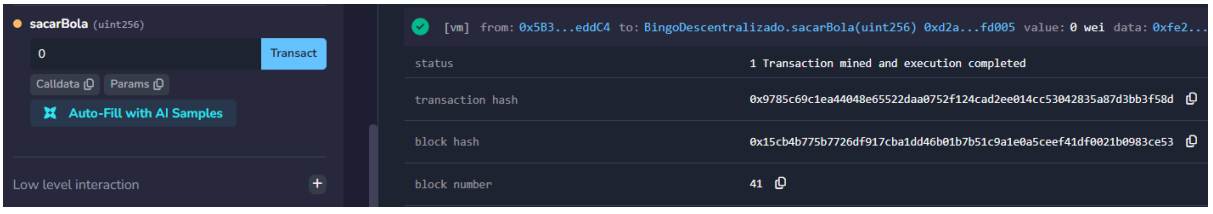


Efectivamente, tras tan solo 3 bloques minados, no se ha llegado al plazo de 5 establecido, por lo que la semilla no puede ser revelada.

A continuación, se procede con la **extracción de bolas** y con la comprobación de **canto de bingo**. En el caso de la partida recién creada con *id=1*, esta no se encuentra en la fase *EnCurso* puesto que su semilla todavía no ha sido revelada, por tanto, no es posible extraer bolas de esta partida:

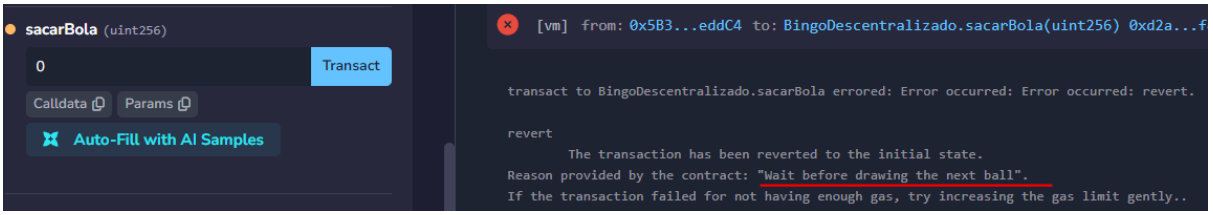


En cambio, en la partida con *id=0*, ya en el estado de *EnCurso*, sí es posible extraerla. Así, se extrae en el bloque nº41 la bola nº60:

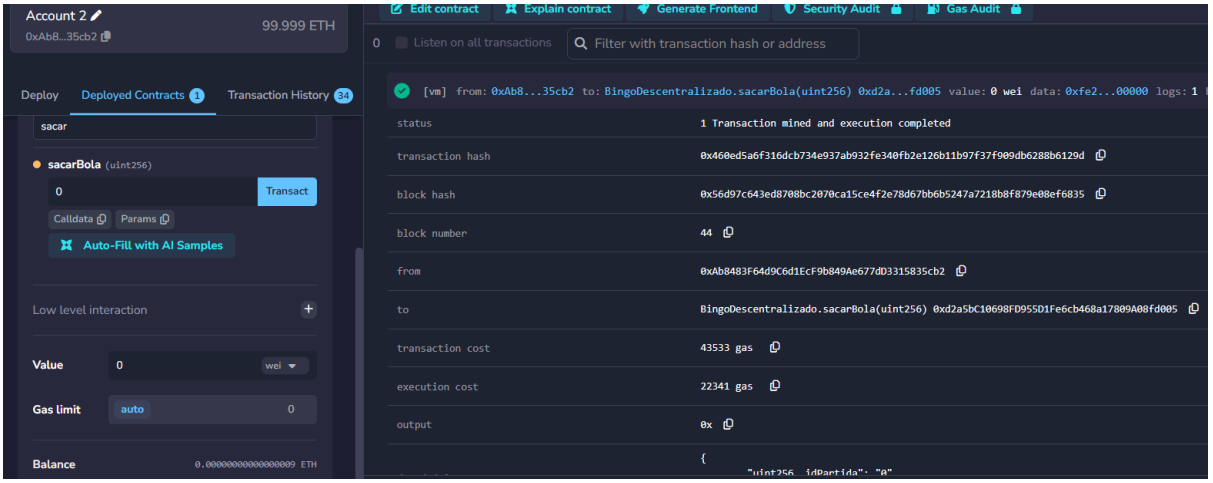



```
[
	{
		"from": "0xd2a5bC10698FD955D1Fe6cb468a17809A08fd005",
		"topic": "0x850853d12a0e72cb1502a94737ccb342a720aa010c58266e2ace60420b5e01c8",
		"event": "BolaExtraida",
		"args": {
			"0": "0",
			"1": "60",
			"2": "1"
		}
	}
]
```

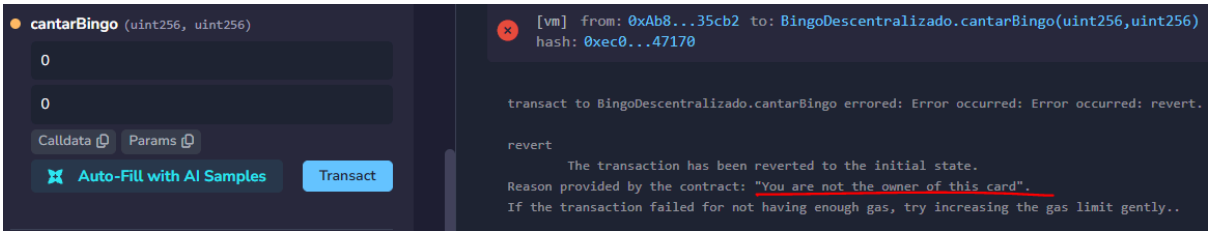
Si se trata de extraer otra bola sin antes haber transcurrido *MIN_BLOQUES_ENTRE_BOLAS* establecido, que era de 3 bloques entre bolas, no se podrá realizar otra extracción:

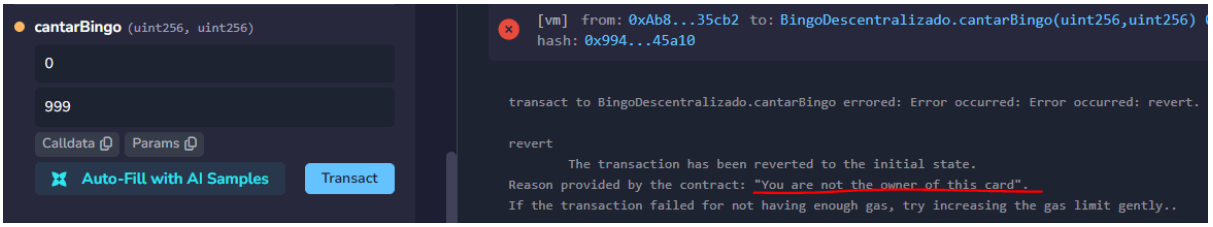


Es decir, entre bola y bola deben minarse 2 bloques para dar tiempo a los jugadores a comprobar el cartón y cantar bingo. Cuando esos bloques entre medias han transcurrido, cualquier jugador puede llamar a *sacarBola()*, no solo el organizador, tal como muestra la siguiente captura con esta llamada desde *Account 2*:

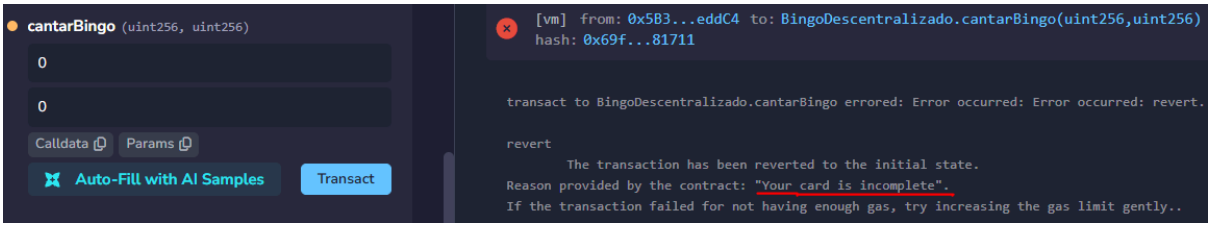


En este momento en el que empiezan a salir los números de cada bola un jugador podría tratar de cantar bingo. Sin embargo, si trata de cantar bingo con cartón que no le pertenece (*Account 2)* o que no existe (*id=999)*, obtiene un error:





Aunque sea el dueño y el cartón exista, es evidente que con tan solo 3 bolas extraídas, no lo tiene completo:



¡Juguemos, pues, al bingo!

Nos centraremos en la partida con *id=0* y en el cartón con *id=0*. Nuestros números son: 2,9,10,**13**,25,26,32,40,41,47,**49**,73,83,84,**85**.

Sacamos bolas hasta llegar a las 15 extraídas para ver qué suerte estamos teniendo con este cartón. Para ver el número de las bolas extraídas y no la máscara en bits con la que el contrato trabaja internamente, podemos hacer uso de la función *verBolasExtraidas()*:

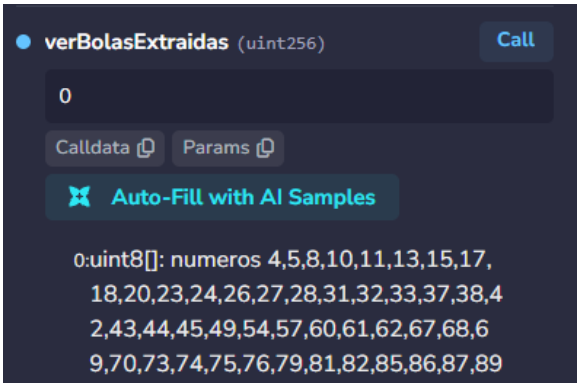


Los 15 números extraídos hasta el momento son: **13**,24,27,28,33,38,44,**49**,60,61,68,76,**85**,87,89.

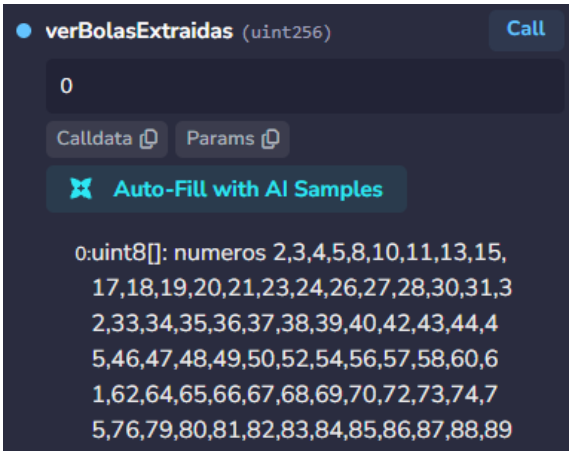
Tan solo 3 números coinciden. Recordemos que esta partida se creó con *X=90*, es decir, obtendremos sí o sí el bonus puesto que antes de las 90 bolas ya habremos cantado bingo. Esto se ha hecho así para comprobar la asignación de porcentajes correctamente.

En un bingo, un cartón suele completarse, de media, entre las 45 y las 75 bolas extraídas.

Vamos a continuar extrayendo bolas hasta llegar a 45:



4,5,8,**10**,11,**13**,15,17,18,20,23,24,**26**,27,28,31,**32**,33,37,38,42,43,44,45,**49**,54,57,60,
61,62,67,68,69,70,**73**,74,75,76,79,81,82,**85**,86,87,89.
Tan solo tenemos 7 números, necesitamos otros 8.
Extraemos bolas hasta la 70:



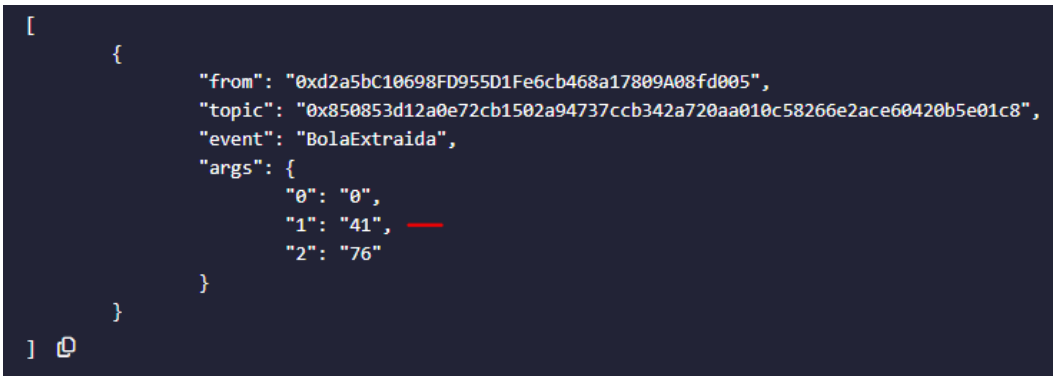
Con 70 números parece difícil no haber podido cantar bingo. Si el jugador, quizá tras una gran cantidad de partidas y dinero “bien jugado”, no está dispuesto a seguir de cerca las bolas extraídas, puede emplear la función *cartonCompleto()* para conocer al instante si su cartón es premiado o no:



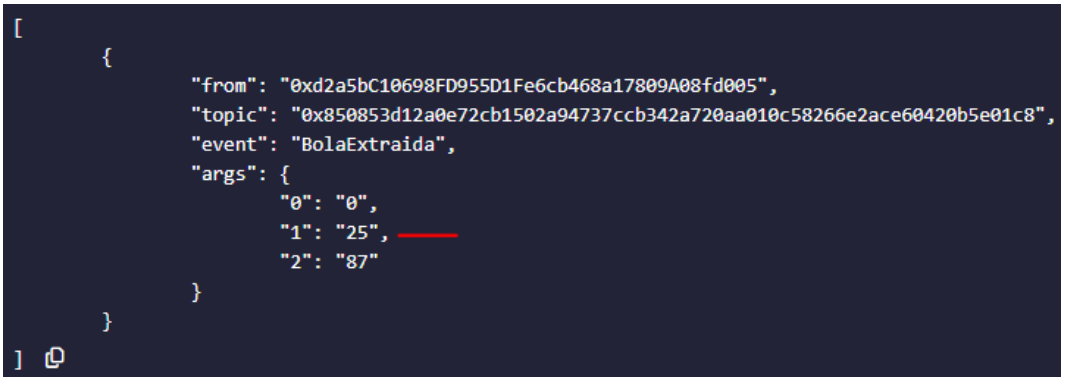
Parece que no es nuestro día de suerte, pues, aparentemente, tras 70 bolas todavía quedan por salir para cantar bingo. Veamos:
2,3,4,5,8,**10**,11,**13**,15,17,18,19,20,21,23,24,**26**,27,28,30,31,**32**,33,34,35,36,37,38,39,
40,42,43,44,45,46,**47**,48,**49**,50,52,54,56,57,58,60,61,62,64,65,66,67,68,69,70,72,**73**,
74,75,76,79,80,81,82,**83**,**84**,**85**,86,87,88,89.

¡Todavía necesitamos 3 números más! Es altamente improbable que, en una partida con más jugadores, no hayamos perdido ya. Aún así, continuaremos con la extracción de bolas hasta que salgan las 3 que nos faltan: 9, 25 y 41…

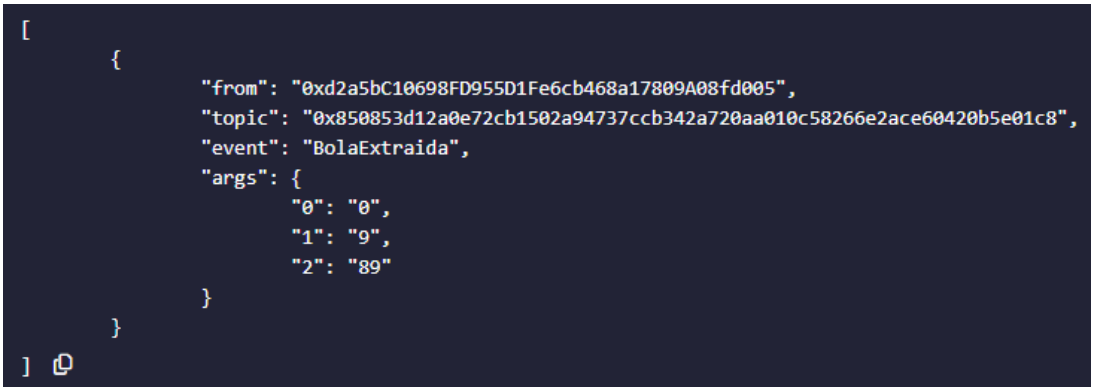
En la extracción nº76 obtenemos el 41:



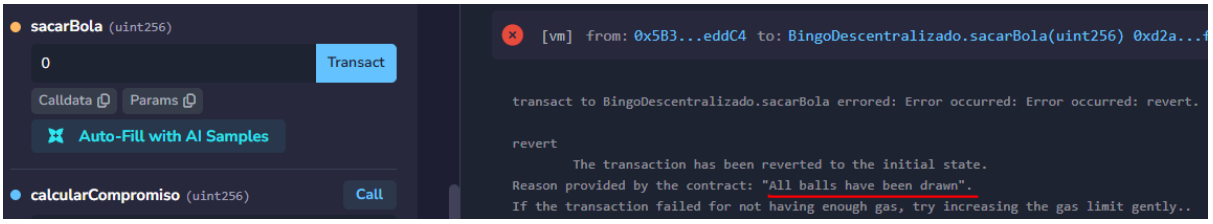
La extracción nº87 ha sido la bola 25:



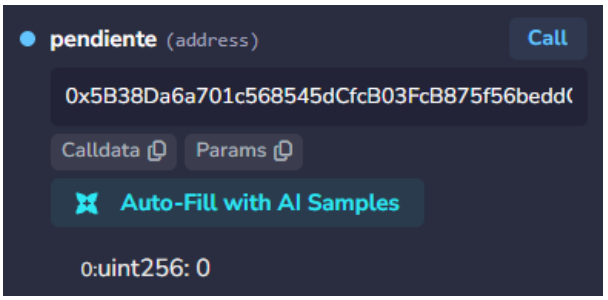
¡Solo quedan 3 bolas y no ha salido el 9! Sin duda, somos unos afortunados.
Finalmente, la penúltima bola extraída es la que nos ha dado el 9:



Por detrás del 9 solo quedaba la 53.
Esto, sin duda, nos está indicando claramente una cosa: ¡debemos jugar otra vez, seguro que la siguiente ganamos antes! Aunque eso ya será tarea de jugadores con mejores capacidades para otro momento.
Por lo pronto, y tras la indudable suerte que hemos tenido con este bingo, podemos alegrarnos de que la comprobación de errores nos impide extraer más bolas (ahora que ya no quedan más):



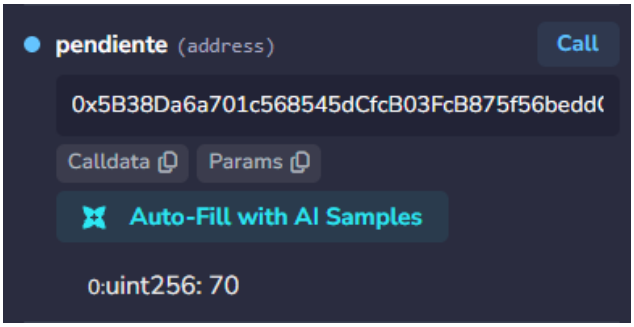
Ahora, podemos cantar el bingo que tanto nos merecemos mediante *cantarBingo()*. Recordemos que el bote de la partida era de 70, y que ahora mismo nuestro saldo en *pendiente* es de 0:



Si todo va bien, y dado que el tope establecido eran 90 bolas, y hemos cantado bingo en la 90 (90 incluida), deberíamos recibir el bonus. Es decir, nos debería pertenecer el 90% del bote (70% base de 70 + 20% bonus de 70 = 63; el 10% restante es la comisión del organizador).



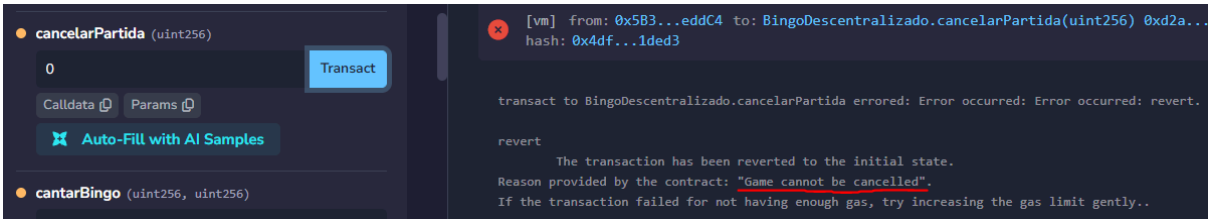
Dado que, en este caso, el jugador de este desastroso cartón fue también el organizador, su saldo en pendiente es de 70, pues obtiene la parte que le corresponde como jugador, pero también la comisión de organizador:



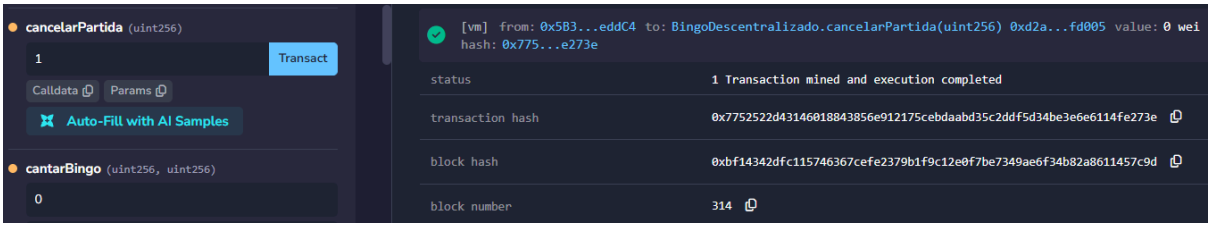
En caso de querer cantar bingo de nuevo, la partida ya ha finalizado y esto no es posible:



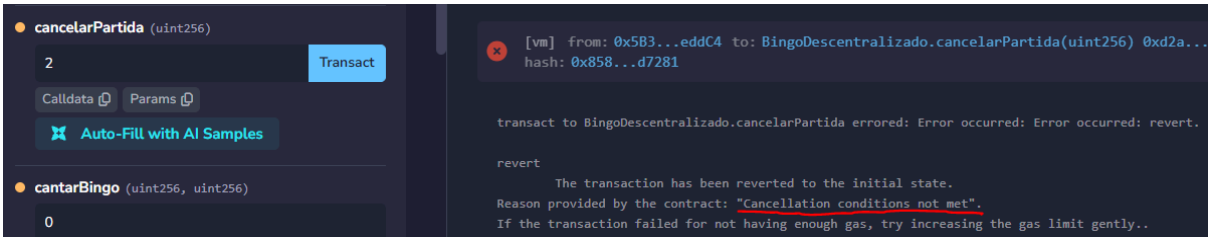
Otra de las funcionalidades que se deben probar son las referentes a la **cancelación de la partida** y el **reembolso** en ellas.
Por ejemplo, si se trata de cancelar una partida que ya ha finalizado, esto evidentemente no se puede llevar a cabo:



En el caso de la partida con *id=1*, la cual cumplía las condiciones de tener el mínimo de cartones (2) y haber agotado la ventana de revelación (200 bloques), la partida es perfectamente cancelable:



Creamos de nuevo la partida que acabamos de cancelar (ahora con *id=2*). Si tratamos de cancelarla nada más crearla obtenemos de nuevo un error pues, aunque no haya cartones comprados y cumpla dicha condición, no cumple la condición restante de no haber transcurrido los bloques establecidos en *plazo* (5) para que la partida se encuentre en el estado de *EnCurso* y haya transcurrido además la ventana de revelación (200 bloques):

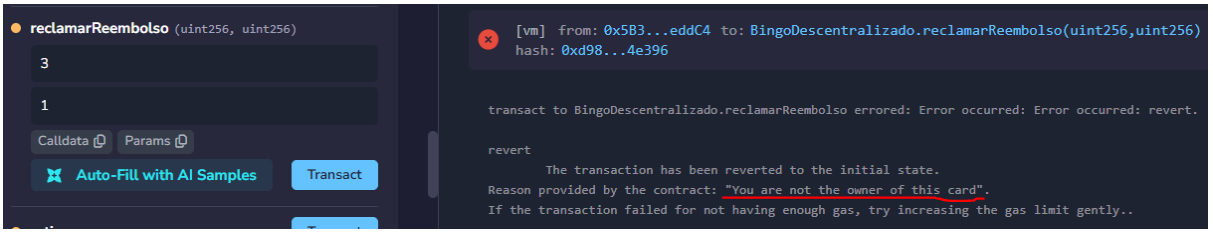


A pesar de adquirir varios cartones en esta partida, no será cancelable hasta que no esté en estado *EnCurso* y transcurra la ventana de revelación. El error recibido será el mismo que el anterior, puesto que estas condiciones son evaluadas en la misma función y arrojan el mismo error.

En cuanto al reembolso, este no se podrá solicitar para una partida que no esté en estado de *Cancelada*, como es el caso de la partida con *id=0*:



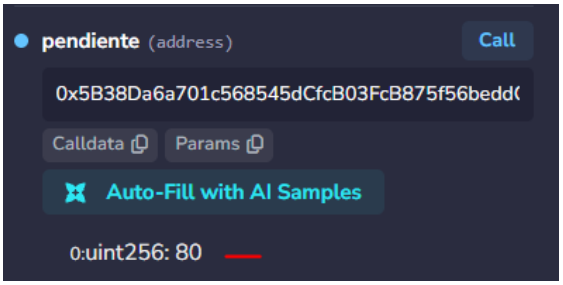
Para probar el reembolso, creamos una nueva partida, está tendrá *id=3*. El organizador (*Account1*) compra un cartón; un segundo jugador (*Account 2*) compra otro cartón. Minamos 200 bloques para que pase la ventana de revelación y cancelamos la partida (puesto que en este tiempo de revelación el organizador no ha revelado la semilla y la partida no está en *EnCurso*).
En este punto, podemos solicitar el reembolso de nuestro cartón. Si probamos con el cartón con *id=1*, perteneciente al segundo jugador, no podemos cancelarlo:



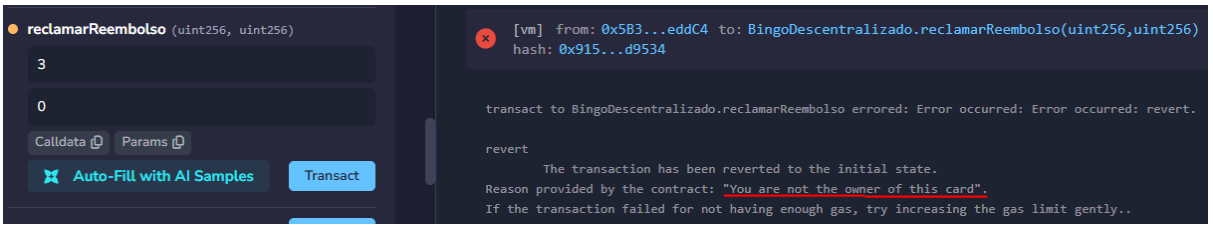
Si seleccionamos nuestro cartón, lo reembolsamos:



Recordamos que antes, en *pendiente*, teníamos un saldo de 70. Ahora, tras el reembolso de este cartón con coste 10, deberíamos tener un saldo de 80:

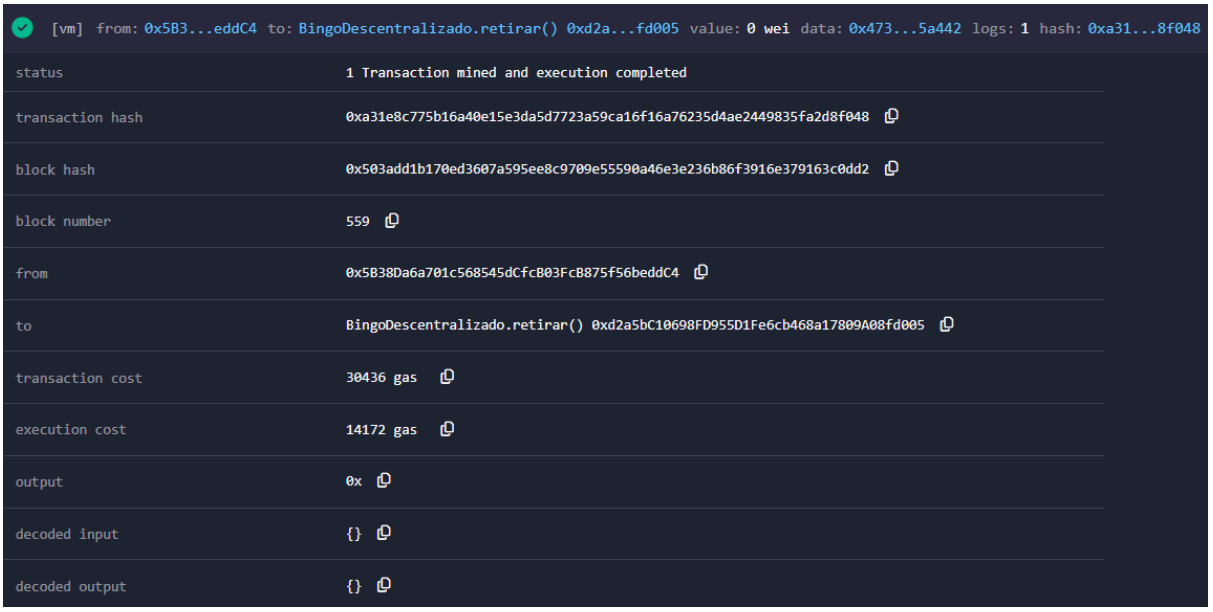


Tampoco es posible reembolsar de nuevo un cartón que ya se ha reembolsado.



El error “You are not the owner of this card” es correcto pues, tal como se estableció, la función *reclamarReembolso* establece como dirección dueño de un cartón reembolsado la dirección “0” (*c.dueno = address(0)*) la cual no existe. Así, el error obtenido es el que corresponde y se evitan ataques de doble reembolso.

Finalmente, llega el momento que todo jugador ansía: la retirada de fondos.
Así, desde la cuenta que ha estado organizando todas estas partidas (*Account 1*), retiramos los fondos que tenemos en nuestro *pendiente* que, ahora mismo, ascienden hasta 80 weis:



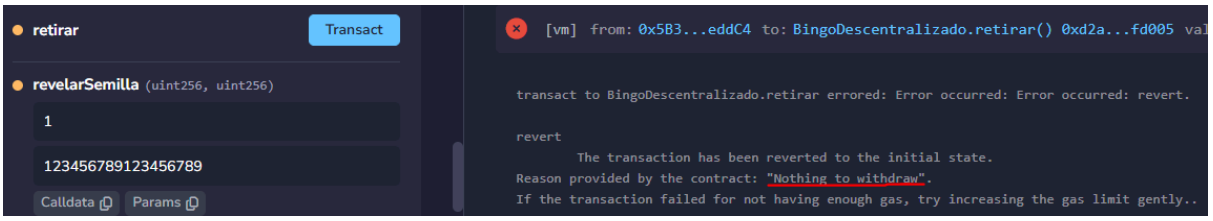
logs


```
[
	{
		"from": "0xd2a5bC10698FD955D1Fe6cb468a17809A08fd005",
		"topic": "0x336c3e0206ecf0ec07f3eb101e022702419f4234101fc9f2dfc056cae287ac73",
		"event": "Retirado",
		"args": {
			"0": "0x5B38Da6a701c568545dCfcB03FcB875f56beddC4",
			"1": "80"
		}
	}
]
```

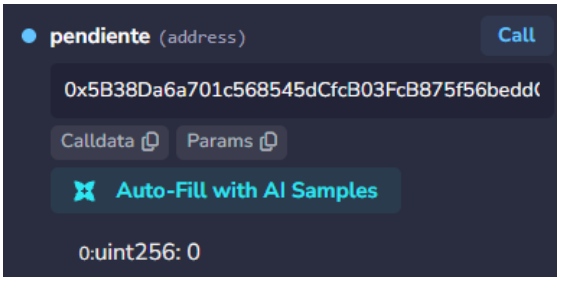

raw logs


```
[
  {
    "_type": "log",
    "address": "0xd2a5bC10698FD955D1Fe6cb468a17809A08fd005",
    "blockHash": "0x503add1b170ed3607a595ee8c9709e55590a46e3e236b86f3916e379163c0dd2",
    "blockNumber": 559,
    "data": "0x0000000000000000000000000000000000000000000000000000000000000050",
    "index": 1,
    "topics": [
      "0x336c3e0206ecf0ec07f3eb101e022702419f4234101fc9f2dfc056cae287ac73",
      "0x0000000000000000000000005b38da6a701c568545dcfcb03fcb875f56beddc4"
    ],
    "transactionHash": "0xa31e8c775b16a40e15e3da5d7723a59ca16f16a76235d4ae2449835fa2d8f048",
    "transactionIndex": 0
  }
]
```


Si tratamos de retirar de nuevo, o si retiramos desde una cuenta sin fondos en *pendiente*, obtenemos el mismo error:



El saldo *pendiente* ha bajado a 0 porque ya ha sido transferido a la cuenta:



Para demostrar que todos los cálculos con los fondos están realizados correctamente, debemos primero todas las transacciones que se han quedado pendientes. Cancelamos las partidas que quedaban por cancelar, reclamamos el reembolso de todas las cuentas, retiramos todo lo *pendiente* de cada cuenta/jugador que se ha empleado durante estas pruebas, y mostramos el saldo del contrato. Si todo ha ido bien, este debería mostrar un saldo de 0:

